

Types and Auto

Cast

- `static_cast` – Compiler-checked, zero cost → CRTP, enum conversion
- `dynamic_cast` – Runtime-checked, vtable lookup → Not used (avoid in embedded)
- `reinterpret_cast` – Unchecked, zero cost → `Reg<Addr> int` → pointer
- `const_cast` – Unchecked, zero cost → can cast away `const` but not used

rvalue and lvalue

- lvalue has a persistent address in mem so can be pointed to
- rvalue is only temporary exists during evaluation
- `& test` : & get the address outof lvalue while rvalue cannot
- `T&` -> lvalue reference -> alias to variable
- `const T&` -> const lvalue reference applicable for rvalue -> read-only alias
- `T&&` -> rvalue reference -> ori value points to nullptr

function pointer

```
/*
 * For historical reasons; programs expect signal's return value to be
 * defined by <sys/signal.h>.
 */
__BEGIN_DECLS
void(*signal(int, void (*)(int)))(int);
__END_DECLS
#endif /* !_SYS_SIGNAL_H_ */
```

- `void(*)(int)` - pointer to a fn taking int returns void -> `fn_ptr`
- signal is inputted int & `fn_ptr` and output `fn_ptr`

Polymorphism

```
class Stm32Gpio : public IGpio {
```

- `public : IGpio& ref = stm32_obj;` → private does not allow this

class, struct and namespace

namespace vs struct

- namespace can be extended anywhere so,
- namespace cannot have `template` parameters, unlike the locally closed struct

```
// valued template !
template<uint32_t Addr>
```

```

struct Reg {
    static volatile uint32_t& ref() {
        return *reinterpret_cast<volatile uint32_t*>(Addr);
    }
    static void set_bits(uint32_t mask) { ref() |= mask; }
    static void clear_bits(uint32_t mask) { ref() &= ~mask; }
};

// Usage
constexpr uint32_t GPIOA_ODR = 0x40020014;
Reg<GPIOA_ODR>::set_bits(1 << 5); // set pin 5 high

```

virtual and override

- virtual prefixed parent method should be overridden by child method
- virtual uint64_t micros() = 0; -> =0 means pure virtual where base cannot provide implementation
- override-suffixed child method lets compiler safety check
 - w/o override may cause wrong parameter
- compiler stores virtual parent fns in vtable
- Cost : child fns points to the parent fns in vtable + indirect call when virtual function call

dependency injection -> polymorphism at consumer

```

// a specific address
constexpr uint32_t GPIOA_ODR = 0x40020014;
Stm32Gpio led(GPIOA_ODR, 5);

```

```

// when Stm32Gpio is the only child
void blink(Stm32Gpio& led) {led.set(true); }
// 1 runtime polymorphism
// vtable lookup, works with any GPIO
void blink(IGpio& led) {led.set(true); }

```

compile time polymorphism

```

// 2 duck-typing compile time polymorphism
// no vtable -> can be constexpr & no base class
// but keep one fns copy per type and no mixed typed arr
template<typename Gpio>
void blink(Gpio& led) {led.set(true); }

// 3 CRTP
// no mixed typed arr
// GpioBase<Stm32Gpio> and GpioBase<NrfGpio> are different essentially
template<typename Derived>
class GpioBase {
public:

```

```

// toggle body is identical for all derived
void toggle() {
    // cast base [this] -> derived [self]
    auto& self = static_cast<Derived&>(*this);
    self.set(!self.read());
}
};

class Stm32Gpio : public GpioBase<Stm32Gpio> {
public:
    void set(bool high) { /* register write */ }
    bool read() { return /* register read */; }
    // toggle() inherited from GpioBase - calls Stm32Gpio::set/read directly
};

```

Stack, Heap, and Object Lifecycle

Fundamentals

reference's hidden pointer and a heap pointer live on the stack.

What is a stack frame?

Every function call pushes a **frame** onto the stack containing:

- Return address (where to go when function ends)
- Arguments passed to the function
- Local variables declared inside the function

When the function returns, the frame is **popped** – all locals are destroyed instantly.

- **Stack** (~1 instruction each, ~100× faster)
 - Allocate: `sub sp, #size`
 - Free: `add sp, #size`
- **Heap** (function call → allocator bookkeeping)
 - Allocate: `bl _malloc` → free-list search, possible `mmap` syscall
 - Free: `bl _free` → coalesce free-list, return block

call chain: `main()` → `foo()` → `bar()`

Stack (grows downward):

	high address
main() frame	x, v, s, gpio
foo() frame	local, temp
bar() frame	i, j ← sp (stack pointer) points here
	low address

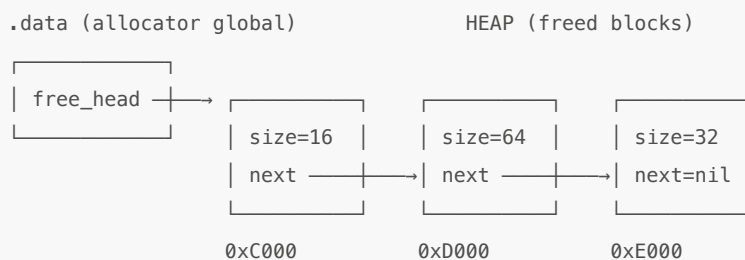
```
bar() returns → sp moves up → bar's frame is gone
foo() returns → sp moves up → foo's frame is gone
```

What is the heap?

Memory pool managed by `malloc/free` (C) or `new/delete` (C++). Allocate/free by user or smart containers (`std::vector<T>`, `std::unique_ptr<T>`).

The allocator (`libmalloc`) maintains a **free-list** – a **linked list** stored inside freed heap blocks:

- **head** pointer: lives in allocator globals (`.data`)
- **next** pointers: live inside each freed block (reusing the now-unused data area)



- `malloc(20)`: walk free-list → find block $\geq 20B$ → split → return pointer
- `free(ptr)`: mark block unused → write **next** pointer into it → re-link list

Constructor and destructor

```
{
    std::vector<int> v = {1, 2, 3};
    // constructor runs: malloc(12), copy {1,2,3} into heap block
    // v.ptr → heap, v.size = 3

    // ... use v ...

} // ← scope ends here
// destructor runs automatically: free(v.ptr)
// heap block is returned to the allocator
```

This is why smart containers (`string`, `vector`, `unique_ptr`) **cannot be memcopy'd** – `memcpy` blindly copies stack bytes (`ptr`, `size`, `cap`), so both `src` and `dest` hold the same heap pointer → both destructors call `free()` on it → double free.

Trivial vs non-trivial types

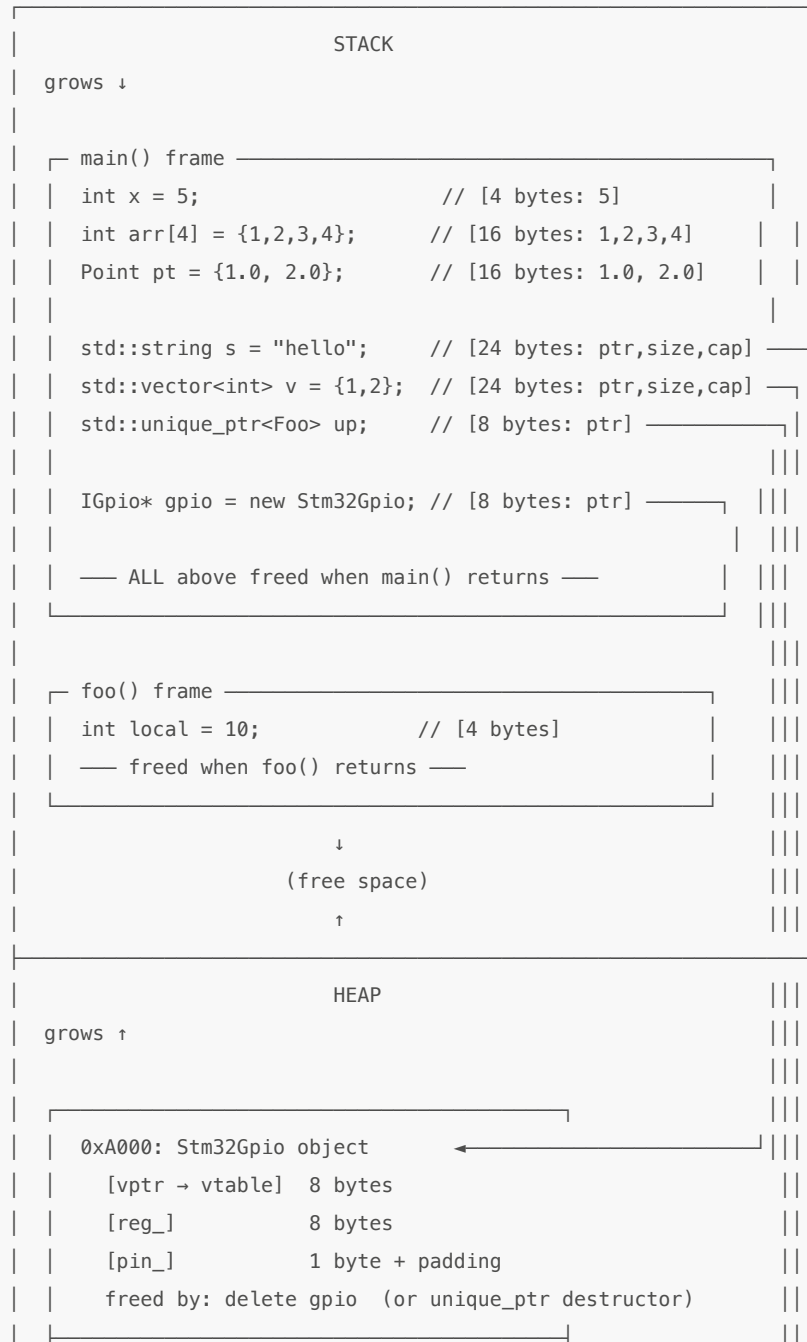
- **Trivial** – no constructor, no destructor, no virtual methods. Raw bytes. `memcpy` is safe.
 - `int`, `double`, `float`, `char`, `int[N]`, `struct { double x, y; }`
- **Non-trivial (resource-managing)** – ctor allocates heap, dtor frees it. `memcpy` → double free.
 - `std::string`, `std::vector<T>`, `std::unique_ptr<T>`
- **Runtime Polymorphism** by **virtual**

- vtable stores the base-to-derived relationship
- base / derived both use vptr to point to their virtual table
- `memcpy` → wrong vptr → virtual calls jump to garbage.

```
static_assert is a compile time check has no burden on runtime
#include <type_traits>
static_assert(std::is_trivially_copyable_v<double>);           // ✓
static_assert(std::is_trivially_copyable_v<int[4]>);           // ✓
static_assert(!std::is_trivially_copyable_v<std::string>);     // ✓ not trivial
```

Full memory layout – where everything lives

High address



```

| | 0xB000: Foo object          ←—————| |
| |   freed by: unique_ptr destructor (automatic) | |
| |—————| |
| | 0xC000: [1, 2] (8 bytes)    ←—————| |
| |   freed by: vector destructor (automatic) | |
| |—————| |
| | 0xD000: "hello\0" (6 bytes) ←—————| |
| |   freed by: string destructor (automatic) | |
| |—————| |
| |
| |
| |.bss (zero-initialized globals/statics) | |
| |   static int count;          // [4 bytes: 0] | |
| |   — lives entire program — | |
| |—————| |
| |.data (initialized globals/statics) | |
| |   int g_mode = 1;           // [4 bytes: 1] | |
| |   static double cal = 3.14; // [8 bytes: 3.14] | |
| |   — lives entire program — | |
| |—————| |
| |.rodata (read-only constants) | |
| |   const char* msg = "hello"; // string literal lives here | |
| |   vtable for Stm32Gpio       // [array of fn pointers] | |
| |   vtable for IGpio          // [array of fn pointers] | |
| |—————| |
| |.text (compiled code) | |
| |   main(), foo(), Stm32Gpio::set(), ... | |
| |—————| |

```

Low address

Copy vs move – by type

Trivial types – memcpy is the entire operation

```

int a = 5;
int b = a;           // compiler emits: memcpy(&b, &a, 4) – done

```

Stack: Stack:

```

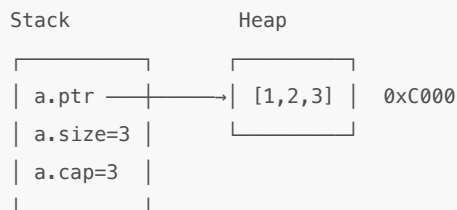
[ a=5 ]  —copy 4B→  [ a=5 ] [ b=5 ]  independent copies

```

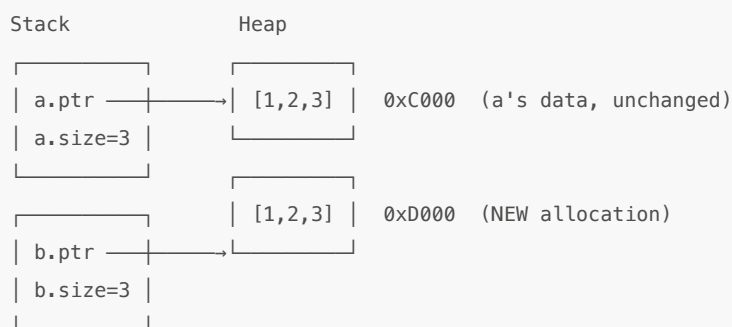
Same for: `double`, `float`, `int[N]`, any struct with no constructor/destructor. No constructor called. No destructor needed. Bitwise copy = correct. `std::move` on trivial types does nothing extra – copy and move are identical.

Non-trivial types – constructor must run, NOT memcpy

```
std::vector<int> a = {1, 2, 3};
```



Copy: `vector<int> b = a;`

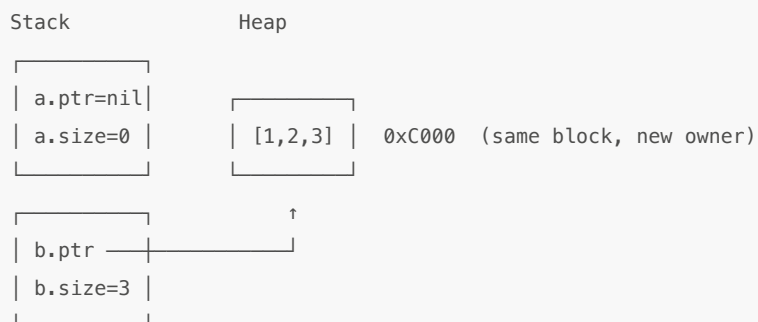


Copy constructor: allocate new heap block, memcpy the CONTENTS

Cost: malloc() + memcpy(N elements)

w/o malloc() allocating new heap block → heap double free

Move: `vector<int> b = std::move(a);`



Move constructor: steal pointer, null source

Cost: 3 pointer/int assignments (24 bytes)

a is still alive and valid – just empty

Virtual method calls – vtable layout

```
IGpio* gpio = new Stm32Gpio(reg, 5);
gpio->set(true); // which set() gets called?
```

- new() -> base ctor links base vtable -> derived ctor overrides with derived vtable

.rodata (compiled into binary, lives forever):

vtable for Stm32Gpio:	
[0] → Stm32Gpio::~~Stm32Gpio()	(.text)
[1] → Stm32Gpio::set()	(.text)
[2] → Stm32Gpio::read()	(.text)

Heap:

vpitr → vtable above	8 bytes (hidden first field)
reg_ = 0x40020014	8 bytes
pin_ = 5	1 byte + 7 padding

gpio->set(true) compiles to:

1. Load vpitr from object (memory read)
2. Load vtable[1] (memory read – get Stm32Gpio::set address)
3. Call that address (indirect branch)

vs. non-virtual call:

1. Call Stm32Gpio::set() (direct branch – address known at compile time)

Cost: 2 extra memory reads + indirect branch ≈ ~5ns overhead

Rules of thumb

- Local variable → stack (automatic, fast, no leak possible)
- Global / static → .data or .bss (lives forever)
- new / malloc → heap (manual or smart-pointer managed)
- String literals ("hello") → .rodata (compiled in, read-only)
- vtables → .rodata (one per class, compiled in)
- Code → .text (compiled in, read-only, executable)
- If a type has a destructor → don't memcpy, use copy/move constructors
- In embedded (MCU) → avoid heap entirely, use static allocation

Operations quick-reference

Pointer vs reference vs value


```
int x = 5;           // value on STACK – owns data directly
int* p = &x;        // pointer on STACK – holds address of x (8 bytes on 64-bit)
int& r = x;         // reference – alias, not a separate object, no extra memory

int* h = new int(10); // pointer on STACK (8 bytes) → data on HEAP (4 bytes)
```

Key distinction: `sizeof(std::vector<int>)` gives the stack footprint (~24 bytes: ptr+size+cap), NOT the heap data size. Use `.size()` for the logical element count.

lvalue vs rvalue

```
int x = 5;           // x is an lvalue – named, has address, persists
                    // 5 is an rvalue – temporary, no lasting identity

process(x);          // binds to int& (lvalue ref)
process(5);           // binds to int&& (rvalue ref)
process(std::move(x)); // std::move CASTS x to rvalue ref – does NOTHING at runtime
                    // it's your move constructor's job to actually steal resources
```

What `memcpy` actually copies

`memcpy` operates on **stack bytes only**. For `std::vector`, those bytes include the internal heap pointer value. After `memcpy`, both objects hold the same pointer → same heap block → double free on destruction.

```
memcpy(&b, &a, sizeof(vector)):
  Copies: [ptr | size | capacity] ← stack bytes (24 bytes)
  Does NOT: allocate new heap, copy elements, or null source
  Result: a.ptr == b.ptr → DOUBLE FREE
```

For `memcpy(dest_buf, src_buf, N * sizeof(int))` (copying between raw heap buffers via `.data()` pointers) – this IS safe because you're copying the actual element data, not the owning object.

`std::move` – what it does and doesn't do

- `std::move(x)` is equivalent to: `static_cast<T&&>(x)`
- casts to rvalue reference
- After `std::move(x)`, `x` is still a valid object in a "moved-from" state (can be destroyed or reassigned, but content is unspecified)

Constructor dispatch

The compiler selects constructors based on value category of the argument:

```
Buffer a(100);           // Buffer(int n)           – normal ctor
Buffer b = a;            // Buffer(const Buffer&) – copy ctor (a is lvalue)
```

```
Buffer c = std::move(a);    // Buffer(Buffer&&)    - move ctor (rvalue ref)
```

const, volatile, inline – compiler/linker effects

Keyword	Affects	Purpose
<code>const</code> (on variable)	compiler	prevents modification, enables placing in <code>.rodata</code>
<code>const</code> (after method)	compiler	method can be called on <code>const&</code> objects; use <code>mutable</code> for exceptions
<code>volatile</code>	compiler	prevents read/write elimination – every access emits a real load/store
<code>inline</code>	linker symbol	changes symbol from strong (T) to weak (W) – allows multiple definitions
<code>constexpr</code>	compiler	enables compile-time evaluation; implies <code>inline</code>
<code>#define</code> MACRO	preprocessor	textual substitution in <code>.ii</code> – no symbol, no type safety

C++ build pipeline

Source Code Stage 1 Stage 2 Stage 3 Stage 4 .cpp → .i/ii → .s → .o → executable
PREPROCESS COMPILE ASSEMBLE LINK

main.cpp main.ii main.s main.o cvd_controller (text) (expanded text) (assembly text) (machine code)
(runnable binary)

Stage 1: PREPROCESS – cpp (or g++ -E)

```
- #include parts of .cpp are expanded by including .h into .ii
- CMake's target_include_directories becomes the -I flag
- cpp -I src/common src/controller/main.cpp -o main.ii
- g++ -E -I src/common src/controller/main.cpp -o main.ii
```

Stage 2: COMPILE – g++ -S

```
- g++ -S -std=c++17 -I src/common src/controller/main.cpp -o main.s
```

Stage 3: ASSEMBLE – g++ -c (or 'as')

```
- g++ -c -std=c++17 -I src/common src/controller/main.cpp -o main.o
```

Stage 4: LINK – g++ (calls ld internally)

```
- g++ main.o -o main
```

All at once

```
- g++ -std=c++17 -I src/common src/controller/main.cpp -o main
```

2. Compilation details

- Lexing & Parser
 - Tokenize source → build Abstract Syntax Tree (AST)
 - tree structure reveals the sequence for execution
- Semantic analysis
 - type checking, access control
 - Verify `pkt.temperature` is double, `send()` args match signature
- Template instantiation
 - replace the `template <typename T> arg`
- Overload resolution
 - `std::cout << pkt.temperature` → pick `operator<<(double)`
- Optimization (-O2)
 - Inline functions, eliminate dead code, unroll loops, vectorize
- Register allocation
 - Map variables to CPU registers (rax, xmm0, etc.)
- Instruction selection
 - Choose x86 instructions (`movsd`, `addsd`, `subsd`)
- Struct layout
 - Compute field offsets: `temperature` is at byte 12 from struct start
- Stack frame layout
 - Decide where each local variable lives relative to Register Stack Pointer

inline and constexpr and Inlining compiler decision

- `inline`, `constexpr`, `template` all produce weak symbols → all ODR-safe in headers
 - each .o carries a copy, linker keeps one, discards duplicates
 - any definition placed in .h MUST be one of these, otherwise ODR violation
- `constexpr` → implies `inline` (weak symbol) + CAN evaluate at compile time
 - `constexpr int x = factorial(5);` → computed by compiler, no runtime call
 - runtime args → falls back to normal (inlined) function call
- Inlining (body expanded at call site, no `bl`) is a compiler optimization
 - -O2 inlines small fns automatically regardless of `inline` keyword
 - too much inlining bloats code → cache misses → slower
 - -O2 also folds obvious compile-time constants → obvious calculation
 - `volatile` enforce compiler to not fold

const mutable volatile atomic

- `const` fns do not change the obj
- `mutable` var of an obj can change under `const` fns
 - [TODO] caching, logging, counters, mutexes
- `volatile` Inform compiler this variable can change outside your view
 - memory-mapped HW registers
 - signal modifying
- `std::atomic` adds thread-safe read/write + memory ordering to `volatile`
- `atomic` is a single CPU instruction `dmb ish` over the complex `std::mutex`
 - `atomic` is `lock_free` while `mutex` is not
 - `dmb ish` drains all pending write before continue
- `atomic` falls back to `mutex` for speed

3. Assemble as

4. Link ld

- Linker resolves compile-yielded symbols by name – it does not re-check types.
- ODR happens when 2 `.o` is compiled with `version-diff` `shared.h`
- `.o` (object file)
 - Symbols: unresolved (U = undefined)
 - Not loadable at runtime
 - Contains one `.cpp`'s machine code
 - Position-independent: not required
 - Used by linker (`ld`) at build time
- `.so` / `.dylib` (shared library)
 - Symbols: all resolved
 - Loadable at runtime (`dlopen` / `ctypes.CDLL`)
 - Contains linked, complete library
 - Position-independent: yes, needs `-fPIC`
 - Used by OS loader at runtime
 - `extern "C"` required for Python/C interop (no mangling)
 - `g++ -std=c++17 -shared -fPIC pid_lib.cpp -o libpid.dylib`

Mangling and symbol

- One executable, one symbol
- Mangling distinguish the over loaded function name in C++ `_ZN3PID7computeEddd || | | ||| || | |`
`|` — d = double (paras) `|| |` — E = end of nested name `||` — 7compute = method name `||` — 3PID = class name `|` — N = nested name start
`|` — _Z = C++ mangled
- `extern "C" void handle_signal(int sig);` disable mangling so the symbol is `handle_signal` only
- `nm main.o`
 - `T` global function / `t` static (internal linkage) function
 - `b` static local data
 - `r` string literals and constants
 - `U` undefined – should be linked at link time
 - `W` weak symbols – inline, `constexpr`, template
- weak symbols

- inline functions are intrinsically weak
- compiler randomly keep one of the multiple conflicting weak symbols

g++ invoke binaries

```
g++ -v -std=c++17 -I src/common src/controller/main.cpp -o main 2>&1 Apple clang version 17.0.0 (clang-1700.6.3.2) Target: arm64-apple-darwin25.4.0 Thread model: posix InstalledDir: /Library/Developer/CommandLineTools/usr/bin
```

-std=c++17 & POSIX & lib c

- std flag specifies which path stds should be found
- POSIX libs are OS specific and can be checked by `g++ -###`
- std:: (C++)
 - Defined by ISO C++ committee
 - Name mangling: yes (`_ZSt5clamp...`)
 - Namespace: `std::`
 - Templates/overloads: yes
 - Talks to kernel: rarely
 - Portable to Windows: yes (MSVC)
- libc (C)
 - Defined by ISO C committee
 - Name mangling: no (`memcpy`)
 - Namespace: `std::` or global
 - Templates/overloads: no
 - Talks to kernel: sometimes (`malloc`)
 - Portable to Windows: yes (MSVC)
- POSIX
 - Defined by IEEE / Open Group
 - Name mangling: no (`signal`)
 - Namespace: global only
 - Templates/overloads: no to support both C and C++
 - Talks to kernel: almost always
 - Portable to Windows: no

struct and class

- Identical except struct is public by default and class is private
- Convention: `struct` for passive data (all public fields, no invariants)
- Convention: `class` when there are invariants, private state, or methods that enforce rules

Headers file

- `<T>` must be in `.h` because compiler needs the body to instantiate per type
 - escape hatch: explicit instantiation in one `.cc` keeps template body out of header
- any non-template definition in `.h` must be marked `inline` to be ODR-safe
 - class member functions defined inside class body are implicitly `inline`
- `#ifndef F00_H_ / #define F00_H_ / #endif` guard prevents multiple inclusion
- one `.h` for all `.cpp` can avoid ODR (One Definition Rule)

- Bazel / CMake guarantees single header version so ODR is avoided

unity build

__restrict

- these pointers do not reference one another during function call
- donot bl hence compiler friendly

```
int getaddrinfo(const char * __restrict, const char * __restrict, const struct addrinfo * __restrict,  
struct addrinfo ** __restrict);
```